

T6 - Verificacion y Pruebas de Seguridad: SAST, DAST, IAST

Fuente: T6-Verificacion y Pruebas de Seguridad-ISS-25-26.pdf
Curso 2025-2026 - Ingenieria del Software Seguro

Indice

- [1. Idea general del tema](#)
 - [2. Por que son necesarias las pruebas de seguridad](#)
 - [3. Tipos de pruebas de seguridad de software](#)
 - [4. SAST](#)
 - [5. DAST](#)
 - [6. IAST](#)
 - [7. Comparacion SAST DAST IAST](#)
 - [8. Monitorizacion](#)
 - [9. Arquitecturas de monitorizacion multinivel](#)
 - [10. Chuleta final de estudio](#)
-

1. Idea general del tema

La **verificacion y las pruebas de seguridad** sirven para comprobar que un sistema no solo funciona, sino que tambien resiste errores, ataques, malas configuraciones y usos no previstos.

La idea central del tema es:

- Todo software crea superficie de ataque.
- La seguridad no debe aparecer solo al final del desarrollo.
- El diseno seguro reduce riesgos, pero no basta.
- Hay que combinar tecnicas: analisis estatico, pruebas dinamicas, pruebas interactivas, revision manual, analisis de dependencias, configuracion segura y monitorizacion.

En este tema se estudian especialmente:

Tecnica	Momento principal	Que analiza	Vision
SAST	Desarrollo temprano	Codigo fuente, bytecode o binarios	White-box
DAST	Aplicacion ejecutandose	Comportamiento externo	Black-box
IAST	Aplicacion ejecutandose con instrumentacion	Codigo y flujos reales	Hibrida
Monitorizacion	Ejecucion y operacion	Eventos, reglas y comportamiento	Continua

La conclusion practica es que ninguna tecnica cubre todo. La estrategia robusta consiste en combinarlas en el ciclo de vida del software.

2. Por que son necesarias las pruebas de seguridad

2.1 Las amenazas afectan a cualquier sistema

Los ataques ya no se dirigen solo a bancos, hospitales, defensa o sistemas claramente criticos. Cualquier aplicacion puede ser un punto de entrada.

Ejemplos:

- Una aplicacion local pequena puede exponer datos de usuarios.
- Un blog vulnerable a SQL injection puede filtrar su base de datos.
- Un dispositivo IoT vulnerable puede servir como entrada a una red corporativa.
- Una dependencia vulnerable puede afectar a miles de sistemas, como ocurrio con Log4j.

La idea importante es que los atacantes suelen aprovechar el eslabon mas debil. Un sistema pequeno puede convertirse en el punto inicial de un ataque mayor.

2.2 Cumplimiento legal y responsabilidad

Las normativas de proteccion de datos obligan a proteger informacion sensible incluso en sistemas pequenos.

Ejemplos de marcos normativos o estándares mencionados:

- **GDPR** en la Union Europea.
- **CCPA** en California.
- **LPRPDE** en Canada.
- **HIPAA, PCI-DSS o ISO 27001** en determinados contextos.

El incumplimiento puede implicar sanciones, responsabilidad legal y perdida de confianza.

2.3 Confianza del consumidor

La seguridad es una expectativa basica. Una brecha de seguridad puede afectar directamente a:

- Reputacion.
- Confianza del usuario.
- Continuidad del negocio.
- Relacion con clientes y proveedores.

Por eso la seguridad tambien puede verse como una ventaja competitiva: un sistema que demuestra buenas practicas transmite confianza.

2.4 Prevenir cuesta menos que corregir

Corregir vulnerabilidades despues del despliegue puede costar mucho mas que corregirlas durante el desarrollo.

La asignatura insiste en el enfoque **shift-left**:

- Introducir seguridad cuanto antes.
- Automatizar deteccion temprana.
- Integrar herramientas en el flujo de desarrollo y CI/CD.
- Evitar que errores simples lleguen a produccion.

Ejemplo claro: credenciales hardcodeadas en un MVP. Si se detectan con SAST antes de desplegar, la correccion es barata; si se explotan en produccion, el impacto puede ser legal, economico y reputacional.

2.5 La seguridad por diseno no es suficiente

El diseno seguro es imprescindible, pero no elimina todos los riesgos.

El documento distingue varias fuentes de problemas:

- Errores de implementacion.
- Errores de configuracion.
- Errores de despliegue.
- Nuevas amenazas que aparecen despues del diseno.
- Dependencias externas vulnerables.

Por eso hacen falta pruebas y verificacion: detectan fallos que no se anticiparon durante el diseno.

3. Tipos de pruebas de seguridad de software

Las pruebas de seguridad no coinciden exactamente con las pruebas funcionales tradicionales. Comparten la idea de encontrar fallos, pero el objetivo especifico es reducir riesgos como:

- Fugas de datos.
- Accesos no autorizados.
- Interrupciones del servicio.
- Escalada de privilegios.
- Explotacion de vulnerabilidades conocidas.
- Errores de configuracion.

3.1 SAST: Static Application Security Testing

SAST analiza codigo fuente, bytecode o binarios sin ejecutar el programa.

Detecta patrones inseguros como:

- SQL injection.
- XSS.
- Uso inseguro de funciones.
- Credenciales hardcodeadas.
- Deserializacion insegura.
- Manejo inseguro de datos.

Es especialmente util en fases tempranas del desarrollo.

3.2 DAST: Dynamic Application Security Testing

DAST analiza la aplicacion en ejecucion desde fuera.

Simula ataques contra:

- Formularios web.
- APIs.
- Flujos de autenticacion.
- Configuracion HTTP/TLS.
- Cabeceras de seguridad.
- Comportamiento observable en runtime.

Se aproxima al punto de vista de un atacante externo.

3.3 IAST: Interactive Application Security Testing

IAST combina ideas de SAST y DAST.

La aplicacion se ejecuta con agentes o instrumentacion que observan:

- Codigo ejecutado.
- Flujos de datos.
- Entradas y salidas.
- Contexto real de la vulnerabilidad.

Suele reducir falsos positivos porque no solo ve codigo sospechoso, sino tambien si ese codigo se ejecuta y como fluye la informacion.

3.4 Pentesting

El **penetration testing** simula ataques reales realizados por expertos.

Caracteristicas:

- Es manual o semiautomatizado.
- Busca explotar vulnerabilidades.
- Evalua impacto real.
- Puede descubrir fallos de logica de negocio dificiles de automatizar.

No sustituye a SAST, DAST o IAST, porque suele ser mas caro y puntual.

3.5 Escaneo de vulnerabilidades

Consiste en usar herramientas automatizadas para detectar vulnerabilidades conocidas en:

- Redes.
- Servidores.
- Sistemas operativos.
- Servicios expuestos.
- Dependencias.

Herramientas típicas:

- Nessus.
- OpenVAS.

3.6 Fuzzing

El **fuzzing** genera entradas aleatorias, inesperadas o malformadas para provocar fallos.

Busca:

- Crashes.
- Excepciones no controladas.
- Fugas de memoria.
- Comportamientos no previstos.
- Posibles vulnerabilidades explotables.

Es muy útil cuando el software procesa formatos, protocolos, parsers o entradas complejas.

3.7 Revision de configuracion de seguridad

Verifica que servidores, bases de datos, redes y servicios estén configurados correctamente.

Ejemplos:

- Permisos adecuados.
- Cifrado activado.
- Firewalls configurados.
- TLS correcto.
- Cabeceras HTTP de seguridad.

- Deshabilitar servicios innecesarios.

3.8 Pruebas de autenticacion y autorizacion

Validan sistemas de identidad y control de acceso.

Aspectos tipicos:

- Login.
- Roles.
- Permisos.
- Gestion de sesiones.
- MFA.
- Bloqueo por intentos fallidos.
- Escalada de privilegios.

3.9 Auditorias de codigo y diseno

Son revisiones manuales o automatizadas de arquitectura, diseno y codigo.

Sirven para encontrar problemas que una herramienta puede no entender bien:

- Flujos de negocio inseguros.
- Suposiciones incorrectas.
- Separacion debil de responsabilidades.
- Fallos de diseno.
- Ausencia de controles en puntos criticos.

3.10 Analisis de dependencias

El analisis de dependencias busca vulnerabilidades conocidas en librerias, APIs y componentes de terceros.

Conceptos importantes:

- **CVE**: identificador de una vulnerabilidad conocida.
- **SCA**: Software Composition Analysis.
- **SBOM**: Software Bill of Materials, inventario de componentes usados por un sistema.

Herramienta mencionada:

- OWASP Dependency-Check.

3.11 Pruebas de cumplimiento

Comprueban si el software satisface obligaciones normativas o estándares.

Ejemplos:

- GDPR.
 - HIPAA.
 - PCI-DSS.
 - ISO 27001.
-

4. SAST

4.1 Definicion

SAST es una tecnica que identifica vulnerabilidades sin ejecutar el software.

Puede analizar:

- Código fuente.
- Bytecode.
- Binarios.

Su objetivo es localizar patrones inseguros antes de que el código llegue a producción.

4.2 Características principales

- Se realiza en fases tempranas del ciclo de desarrollo.
- Encaja con el enfoque **shift-left**.
- Puede integrarse en IDEs, repositorios y pipelines CI/CD.
- Ayuda a cumplir estándares como OWASP Top 10 y CWE.
- Detecta problemas repetibles mediante reglas.

4.3 Tipos de análisis estático

Tipo	Que examina	Ejemplo	Ventaja
Codigo fuente	Codigo escrito por el equipo	Bandit, SonarQube	Detecta fallos desde el desarrollo
Bytecode	Codigo compilado intermedio	SpotBugs en Java	Util si no se analiza directamente el fuente
Binarios	Ejecutables o librerias compiladas	Analisis de ejecutables	Util en entornos restrictivos

4.4 Herramientas SAST para Python

Herramienta	Uso principal
Bandit	Detecta vulnerabilidades comunes en Python: comandos inseguros, SQL injection, secretos hardcodeados, etc.
Pylint con plugins de seguridad	Busca malas practicas y usos inseguros, por ejemplo modulos peligrosos.
Safety	Escanea dependencias de <code>requirements.txt</code> contra CVEs y bases de datos de vulnerabilidades.
Semgrep	Busca patrones peligrosos mediante reglas personalizables y soporta varios lenguajes.
Checkmarx	Solucion comercial para analisis profundo y seguimiento de flujos de datos.
SonarQube con SonarPython	Plataforma de calidad y seguridad con dashboard, reglas y analisis de bugs, vulnerabilidades y code smells.
SonarQube for IDE / SonarLint	Detecta problemas en el editor en tiempo real y puede sincronizar reglas con SonarQube Server.

Reglas o problemas tipicos en Python:

- Uso inseguro de `pickle`.
- Uso inseguro de `yaml.load()`.
- Credenciales hardcodeadas.
- Riesgos en Django o Flask.
- Posible CSRF.
- Posible XSS.

- Dependencias vulnerables.

4.5 Herramientas SAST para Java

Herramienta	Uso principal
SpotBugs	Detecta errores comunes y patrones peligrosos en bytecode Java.
PMD	Analiza malas practicas y problemas de codigo.
Checkmarx	Escaneo comercial para entornos empresariales.
SonarQube	Plataforma integral con reglas configurables de calidad y seguridad.

4.6 Importancia de SAST

SAST es importante porque:

- Detecta vulnerabilidades pronto.
- Reduce coste de remediacion.
- Automatiza revisiones en CI/CD.
- Ayuda al cumplimiento normativo.
- Educa al desarrollador al mostrar problemas cerca del codigo.
- Previene riesgos antes de desplegar.

4.7 Ejemplo: SQL Injection en Python

Caso vulnerable:

```
query = f"SELECT * FROM users WHERE username = '{user_input}'"
```

Problema:

- Se concatena entrada de usuario dentro de una consulta SQL.
- Un atacante puede modificar la consulta.

SAST podria alertar:

- Bandit: posible vector de SQL injection.
- Reglas de seguridad Python: posible SQL injection en string format.

Solucion:

- Usar consultas parametrizadas.
- Usar ORM correctamente.
- No concatenar entrada no confiable.

4.8 Ejemplo: credenciales hardcodeadas

Caso vulnerable:

```
def bad_api_key_handling() -> str:
    api_key = "1234-abcd-5678-efgh"
    return api_key
```

Problema:

- La clave queda expuesta en el repositorio.
- Puede filtrarse por commits, logs, copias o despliegues.

Soluciones:

- Variables de entorno.
- Gestores de secretos.
- Ficheros protegidos fuera del repositorio.
- Rotacion de claves si se han filtrado.

4.9 Ejemplo: XSS en Flask

Caso vulnerable:

```
@app.route("/search")
def search():
    query = request.args.get("q")
    return f"Resultados para: {query}"

if __name__ == "__main__":
    app.run(debug=True)
```

Problemas:

- Entrada de usuario devuelta sin escapar.
- `debug=True` expone el debugger y puede permitir ejecucion de codigo en determinados escenarios.

Soluciones:

- Escapar la salida.
- Usar plantillas seguras.
- Sanitizar entradas cuando proceda.
- Desactivar modo debug en produccion.

4.10 Ejemplo: deserializacion insegura

Caso vulnerable en Java:

```
ObjectInputStream ois = new ObjectInputStream(inputStream);  
Object obj = ois.readObject();
```

Problema:

- Si los datos no son confiables, la deserializacion puede provocar ejecucion de codigo o manipulacion de objetos.

Soluciones:

- No deserializar datos no confiables.
- Validar entradas.
- Usar formatos mas seguros como JSON.
- Aplicar listas permitidas de tipos si la deserializacion es imprescindible.

4.11 Ventajas de SAST

- Muy eficiente si la regla detecta el problema.
- Da respuesta temprana.
- Se integra bien en IDE y CI/CD.
- Reduce coste de correccion.
- Mejora la formacion del equipo.
- Ayuda a detectar problemas repetidos en todo el codigo.

4.12 Limitaciones de SAST

- Puede generar falsos positivos.
- Puede generar falsos negativos.
- No siempre propone soluciones adecuadas.
- Requiere configuracion.

- Necesita conocimiento para interpretar resultados.
- No ve bien problemas de configuracion real de produccion.
- No siempre entiende logica de negocio.

4.13 Recomendaciones para usar SAST

- Usar mas de una herramienta cuando el riesgo lo justifique.
 - Ajustar reglas al proyecto.
 - Integrarlo en IDE y CI/CD.
 - Revisar resultados con criterio, no aceptar todo automaticamente.
 - Priorizar vulnerabilidades explotables y de alto impacto.
 - Combinarlo con DAST, IAST y revision manual.
-

5. DAST

5.1 Definicion

DAST analiza una aplicacion mientras se esta ejecutando. La herramienta interactua con la aplicacion desde fuera, como si fuera un cliente o atacante.

No necesita acceder al codigo fuente. Por eso se considera un enfoque **black-box**.

5.2 Que detecta DAST

DAST puede encontrar problemas que SAST no ve bien:

- Configuraciones inseguras del servidor.
- Cabeceras HTTP ausentes.
- Problemas TLS.
- XSS observable en runtime.
- SQL injection en APIs.
- Autenticacion debil.
- Ausencia de rate limiting.
- Errores de autorizacion.
- Exposicion de datos en respuestas.
- Flujos explotables en la aplicacion desplegada.

5.3 Importancia de DAST

DAST es importante porque prueba el sistema en condiciones mas cercanas a la realidad.

Ventajas:

- Detecta fallos en produccion o entornos similares.
- Encuentra problemas de configuracion.
- Simula ataques externos.
- Complementa SAST.
- Ayuda a comprobar cumplimiento de OWASP ASVS o PCI DSS.

5.4 Ejemplo: deteccion de XSS

Escenario:

- Aplicacion web con formulario de busqueda.
- La entrada no se sanitiza correctamente.
- SAST no lo detecto porque el flujo estaba oculto o era dificil de analizar.

Ataque simulado:

```
<script>alert('XSS')</script>
```

Resultado:

- Si el script se ejecuta, DAST reporta XSS.

Soluciones:

- Escapar salida.
- Sanitizar entrada cuando sea necesario.
- Usar Content-Security-Policy.
- Evitar renderizar HTML no confiable.

5.5 Ejemplo: SQL Injection en API

Escenario:

- API REST con parametro `id` no sanitizado.

- La API puede ser de un tercero, por lo que SAST propio no puede analizar su código.

Ataque:

```
id=1' OR '1'='1
```

Resultado:

- Si la API devuelve datos de múltiples usuarios o una respuesta inesperada, la herramienta alerta.

Soluciones:

- Consultas parametrizadas.
- ORM bien usado.
- Validación de entradas.
- Principio de mínimo privilegio en la base de datos.

5.6 Ejemplo: autenticación débil

Escenario:

- Panel de administración sin protección contra fuerza bruta.
- Las pruebas de desarrollo no detectaron el problema.

Ataque simulado:

- Probar combinaciones comunes como `admin/admin123`.

DAST puede detectar:

- Falta de límite de intentos.
- Ausencia de MFA.
- Respuestas que facilitan enumeración de usuarios.

Soluciones:

- MFA.
- CAPTCHA cuando proceda.
- Bloqueo temporal.
- Rate limiting.

- Alertas ante intentos anormales.

5.7 Ejemplo: configuracion insegura

Escenario:

- Servidor sin cabeceras de seguridad.
- Configuracion de desarrollo distinta a produccion.

DAST puede detectar:

- Falta de `X-Content-Type-Options` .
- TLS debil.
- Cabeceras ausentes.
- Politicas inseguras.

Soluciones:

- Hardening del servidor.
- Cabeceras HTTP seguras.
- Configuracion TLS moderna.
- Revisar diferencias entre DEV y PROD.

5.8 Ejemplo: flujos de trabajo mal configurados

El documento presenta un problema de pipeline:

- La misma maquina o agente de compilacion genera builds de DEV y PROD.
- Los desarrolladores tienen permiso para lanzar builds DEV, pero no PROD.
- Si un atacante compromete una cuenta de desarrollador, puede lanzar una build DEV maliciosa.
- Esa build puede infectar el agente de compilacion.
- Luego el agente puede afectar a una build de produccion.

Leccion:

- Los entornos y agentes de build deben aislarse.
- No debe confiarse ciegamente en codigo de origen desconocido.
- Es importante separar privilegios entre DEV y PROD.
- Los IDEs preguntan si confias en el codigo por este tipo de riesgos.

5.9 Como funciona DAST

De forma simplificada:

1. La herramienta identifica puntos de entrada y salida.
2. Asocia reglas o pruebas a esos flujos.
3. Ejecuta la aplicacion enviando entradas de prueba.
4. Captura respuestas.
5. Analiza si las respuestas violan reglas de seguridad.
6. Genera un informe con resultados.

5.10 Limitaciones de DAST

- Puede no cubrir rutas que no alcance durante el escaneo.
- Puede necesitar credenciales o configuracion de autenticacion.
- Puede generar ruido.
- Puede ser lento.
- No siempre localiza la linea exacta del codigo vulnerable.
- Puede ser peligroso contra entornos de produccion si las pruebas son agresivas.

5.11 Recomendaciones para usar DAST

- Ejecutarlo en entornos controlados o staging.
 - Configurar usuarios de prueba y datos no sensibles.
 - Incluir rutas autenticadas.
 - Evitar pruebas destructivas en produccion.
 - Revisar resultados junto con logs.
 - Combinarlo con SAST para localizar causas en codigo.
-

6. IAST

6.1 Definicion

IAST analiza aplicaciones en tiempo de ejecucion, pero con visibilidad interna gracias a instrumentacion o agentes.

Combina:

- La vision de codigo y flujos de SAST.
- La ejecucion real de DAST.

Por eso se considera una tecnica hibrida.

6.2 Como funciona

Normalmente se instala un agente en la aplicacion o entorno de ejecucion.

Durante pruebas automatizadas o manuales, el agente observa:

- Rutas ejecutadas.
- Metodos invocados.
- Datos de entrada.
- Transformaciones de datos.
- Consultas a base de datos.
- Respuestas generadas.
- Uso de APIs sensibles.

Si detecta que una entrada no confiable llega a un punto peligroso, puede reportar una vulnerabilidad con bastante contexto.

6.3 Caracteristicas clave

- Analiza en tiempo de ejecucion.
- Tiene contexto real.
- Puede detectar vulnerabilidades en flujos complejos.
- Reduce falsos positivos respecto a SAST/DAST tradicionales.
- Encaja con CI/CD y pruebas automatizadas.
- Es especialmente util en aplicaciones criticas.

6.4 Herramientas IAST

Herramienta	Caracteristica mencionada
Contrast Security	Integracion con Java, .NET y Node.js.
Veracode IAST	Funciona con aplicaciones en contenedores o servidores.
Synopsys Seeker	Soporta APIs y aplicaciones web modernas.
Hdiv Detection	Especializado en lenguajes como Python y Ruby.

6.5 Importancia de IAST

Ventajas:

- Menos falsos positivos al observar contexto real.
- Detecta flujos complejos, por ejemplo autenticacion OAuth.
- Da feedback durante pruebas.
- Puede integrarse en pipelines DevOps.
- Ayuda a priorizar porque muestra donde se produce el problema.

Casos de uso:

- Banca.
- Salud.
- Aplicaciones con datos sensibles.
- Equipos que necesitan retroalimentacion inmediata.
- Sistemas con flujos dificiles de comprobar solo con SAST o DAST.

6.6 Ejemplo: SQL Injection en API

Escenario:

- API REST en Java.
- Consulta SQL construida por concatenacion de strings.

Deteccion con IAST:

- Durante una prueba E2E, el agente detecta que una entrada llega a una consulta SQL insegura.
- Puede indicar el metodo vulnerable, por ejemplo
`UserRepository.getUser()` .

Solucion:

- Parametros vinculados.
- JPA/Hibernate correctamente configurado.
- Validacion y control de entradas.

6.7 Ejemplo: XSS en tiempo de ejecucion

Escenario:

- Aplicacion React renderiza datos no sanitizados del usuario.

Entrada de prueba:

```
<img src=x onerror=alert(1)>
```

Deteccion:

- La herramienta observa que la entrada llega al renderizado sin sanitizacion.

Solucion:

- Sanitizar con DOMPurify u otra libreria adecuada.
- Evitar APIs de renderizado inseguras.
- Aplicar Content-Security-Policy.

6.8 Limitaciones de IAST

- Requiere instrumentar la aplicacion.
- Puede tener coste de rendimiento.
- Depende de que las pruebas ejecuten los flujos relevantes.
- Puede ser mas complejo de desplegar.
- Suele depender de herramientas comerciales.

6.9 Recomendaciones para usar IAST

- Integrarlo con pruebas E2E y de integracion.
- Ejecutarlo en entornos de test o staging.
- Asegurar buena cobertura funcional.
- Revisar impacto en rendimiento.
- Usarlo en sistemas donde el contexto de ejecucion sea importante.

7. Comparacion SAST DAST IAST

7.1 Tabla comparativa

Criterio	SAST	DAST	IAST
Necesita ejecutar la app	No	Si	Si
Necesita codigo	Normalmente si	No	No siempre, pero necesita

Criterio	SAST	DAST	IAST
fuelle			instrumentacion
Momento ideal	Desarrollo temprano	Staging, QA, preproduccion o produccion controlada	Tests automatizados o manuales con app ejecutandose
Vision	Interna	Externa	Interna y externa
Tipo	White-box	Black-box	Hibrida
Encuentra bien	Codigo inseguro	Configuracion y fallos explotables en runtime	Vulnerabilidades con contexto real
Localiza causa	Buena	Limitada	Buena
Falsos positivos	Puede tener muchos	Puede tener ruido	Menos que SAST/DAST en muchos casos
Riesgo al ejecutar	Bajo	Medio si ataca sistemas reales	Medio por instrumentacion
Mejor uso	Shift-left, IDE, CI/CD	Validar app desplegada	Validar flujos reales durante pruebas

7.2 Cuando usar cada una

Usa **SAST** cuando:

- Quieres detectar problemas antes de ejecutar.
- El equipo desarrolla codigo propio.
- Necesitas feedback en IDE o pull request.
- Quieres reglas repetibles y automatizadas.

Usa **DAST** cuando:

- Quieres comprobar el comportamiento real de una aplicacion desplegada.
- Te preocupan configuraciones, cabeceras, TLS o flujos web.
- No tienes acceso al codigo fuente.
- Quieres simular ataques externos.

Usa **IAST** cuando:

- Quieres menos falsos positivos.
- Necesitas contexto interno durante pruebas reales.
- Hay flujos complejos.
- Puedes instrumentar la aplicacion.

7.3 Idea clave de examen

No son alternativas excluyentes.

La frase importante seria:

SAST detecta temprano problemas en codigo; DAST valida la aplicacion en ejecucion desde fuera; IAST combina ejecucion real con visibilidad interna para reducir ruido y mejorar precision.

8. Monitorizacion

8.1 Definicion general

La **monitorizacion** o supervision observa el comportamiento de aplicaciones y sistemas durante su ejecucion para detectar desviaciones, fallos o comportamientos inseguros.

Puede servir para:

- Detectar mal comportamiento.
- Reaccionar ante incidentes.
- Detener, reiniciar o cambiar permisos.
- Apoyar la evolucion del software.
- Notificar errores a proveedores o responsables.
- Detectar fallos de implementacion, diseno o interaccion.

8.2 Variantes de supervision

El documento diferencia varias dimensiones:

Dimension	Opciones
Generacion de eventos	Interna o externa
Lugar del monitor	Interno o externo

Dimension	Opciones
Actitud	Observar o actuar
Tiempo de respuesta	Reactivo o predictivo
Tipo de aplicacion	Sin supervision, autosupervisada o instrumentada

8.3 Monitores internos y externos

Monitor interno:

- Se ejecuta en el mismo dominio de confianza que la aplicacion.
- Tiene mas acceso al estado interno.
- Puede ser mas facil de integrar con la logica de la aplicacion.

Monitor externo:

- Se ejecuta en un dominio de confianza diferente.
- Puede ser mas resistente si la aplicacion falla o es comprometida.
- Puede observar desde fuera varias aplicaciones o plataformas.

8.4 Aplicaciones sin supervision

Son aplicaciones que no incluyen mecanismos especificos de supervision.

Ventajas:

- Mas flexibles ante cambios de entorno.
- Aplicables a aplicaciones heredadas.
- No requieren modificar la aplicacion.

Inconvenientes:

- Menos eficaces.
- Usan especificaciones genericas.
- Mas propensas a errores.
- Problemas de responsabilidad: puede ser dificil saber quien debe reaccionar o corregir.

8.5 Aplicaciones autosupervisadas

Incluyen codigo interno de autocomprobacion.

Ejemplo conceptual:

- Una aplicacion que genera logs de seguridad y comprueba reglas para evitar situaciones no deseadas.

Ventajas:

- Transparente para la plataforma.
- Menor probabilidad de introducir vulnerabilidades en la plataforma.
- Conoce bien su propio estado.

Inconvenientes:

- Requiere esfuerzo adicional de programacion.
- Es costoso cambiar reglas con el sistema en funcionamiento.
- Algunas comprobaciones no son posibles, por ejemplo ciertas interacciones externas.
- La capacidad de reaccion puede ser limitada.
- La plataforma no controla completamente las comprobaciones internas.

8.6 Aplicaciones instrumentadas

Son aplicaciones a las que se añade codigo especifico para supervision:

- Generadores de eventos.
- Capturadores de eventos.
- Reglas de monitorizacion.
- Integracion con monitores externos.

Normalmente el supervisor es externo.

Ventajas:

- Mayor tolerancia a fallos y ataques.
- Supervision ajustada a la aplicacion.
- Reglas especificas.
- Capacidad para observar eventos internos.

Inconvenientes:

- Mayor coste.
- Adaptabilidad limitada.
- La autenticidad de los eventos puede ser problematica.

8.7 Cuando es necesaria la instrumentacion

La instrumentacion puede ser necesaria cuando:

- Hay varias instancias de la misma aplicacion en entornos distintos.
- Hay ejecuciones en maquinas virtuales o entornos fisicos diferentes.
- Se necesitan eventos internos fiables.
- Se quiere detectar cambios de estado internos, por ejemplo de `estado1` a `estado2`.

Esquemas posibles:

- Instrumentacion del codigo fuente.
 - Instrumentacion del ejecutable.
 - Instrumentacion automatica.
 - Instrumentacion asistida.
 - Instrumentacion manual.
-

9. Arquitecturas de monitorizacion multinivel

9.1 Idea general

La monitorizacion simple y local puede ser limitada. Para sistemas complejos, el tema propone arquitecturas distribuidas y multicapa.

Estas arquitecturas permiten supervisar:

- Comportamiento interno de una aplicacion.
- Interacciones entre aplicaciones.
- Fallos de diseno.
- Fallos de implementacion.
- Comportamientos anormales en entornos virtualizados.

9.2 Componentes principales

Componente	Nombre completo	Tambien llamado	Que supervisa
LAS	Local Application Surveillance	Monitor local	Reglas especificas de una aplicacion

Componente	Nombre completo	Tambien llamado	Que supervisa
IPS	Intra-Platform Surveillance	Monitor horizontal	Interacciones inesperadas entre aplicaciones
GAS	Global Application Surveillance	Monitor vertical	Reglas especificas de aplicacion a nivel global

9.3 LAS: Local Application Surveillance

LAS supervisa reglas especificas de la aplicacion.

Objetivo:

- Detectar comportamientos inesperados resultantes de fallos de implementacion.

Modelo asociado:

- **ABSM**: Application Behaviour Security Model, modelo de seguridad del comportamiento de la aplicacion.

Ejemplo:

- Una aplicacion no deberia pasar de un estado interno a otro sin una autorizacion concreta.
- Si ocurre, LAS puede detectar la violacion de regla.

9.4 IPS: Intra-Platform Surveillance

IPS supervisa interacciones dentro de una plataforma.

Tambien se llama:

- Monitor horizontal.

Objetivo:

- Detectar interacciones inesperadas entre aplicaciones.

Modelos asociados:

- **AIMS**: Application Interaction Security Model.
- **MFC**: Monitoring Forwarding Control.

Ejemplo:

- Una aplicacion intenta comunicarse con otra de una forma no permitida por el modelo.

9.5 GAS: Global Application Surveillance

GAS supervisa reglas especificas de una aplicacion desde una perspectiva global.

Tambien se llama:

- Monitor vertical.

Objetivo:

- Detectar fallos de diseno de la aplicacion.

Modelo asociado:

- Tipo extendido de ABSM.

Ejemplo:

- Varias instancias distribuidas de una aplicacion incumplen una regla global que no se ve desde una sola instancia.

9.6 Arquitectura NOVA

La arquitectura NOVA presentada en el documento combina:

- Varias aplicaciones.
- Varias maquinas virtuales.
- Monitores LAS.
- Monitores IPS.
- Monitores GAS.
- Proveedores de aplicacion o de maquina virtual.

La idea es distribuir la supervision por capas:

- LAS observa comportamiento local.
- IPS observa relaciones dentro de la plataforma.
- GAS observa reglas globales o verticales.

9.7 Funcionalidades de la monitorizacion

La monitorizacion permite:

- Detectar mal comportamiento de aplicaciones.
- Reaccionar ante ese comportamiento.
- Detener una aplicacion.
- Restablecer un componente.
- Cambiar recursos.
- Cambiar prioridades.
- Cambiar permisos.
- Apoyar la evolucion del software.
- Detectar errores en el modelo de aplicacion.
- Detectar errores de implementacion.
- Detectar errores por interacciones inesperadas.
- Notificar errores a proveedores.
- Mantener control por parte de los usuarios.

9.8 Conclusiones sobre monitorizacion

Ideas clave:

- La supervision sera esencial en sistemas de seguridad futuros.
- La supervision simple y local tiene utilidad limitada.
- La monitorizacion distribuida multicapa permite funcionalidades avanzadas.
- La monitorizacion complementa las pruebas, porque observa el sistema durante su vida operativa.

10. Chuleta final de estudio

10.1 Definiciones rapidas

Concepto	Definicion corta
Verificacion de seguridad	Comprobar que el software cumple propiedades y requisitos de seguridad.
Pruebas de seguridad	Actividades para encontrar vulnerabilidades, malas configuraciones o riesgos explotables.

Concepto	Definicion corta
Shift-left	Mover seguridad a fases tempranas del ciclo de desarrollo.
SAST	Analisis estatico sin ejecutar la aplicacion.
DAST	Analisis dinamico de la aplicacion en ejecucion desde fuera.
IAST	Analisis interactivo en ejecucion con instrumentacion interna.
SCA	Analisis de componentes y dependencias de terceros.
SBOM	Inventario de componentes software usados por el sistema.
Fuzzing	Pruebas con entradas aleatorias o malformadas.
Pentesting	Simulacion de ataques reales por expertos.
Monitorizacion	Observacion continua de comportamiento y eventos durante ejecucion.

10.2 Preguntas tipicas y respuestas

Por que no basta con seguridad por diseno?

Porque pueden aparecer errores de implementacion, configuracion, despliegue, dependencias vulnerables y nuevas amenazas que no se anticiparon durante el diseno.

Que diferencia principal hay entre SAST y DAST?

SAST analiza codigo sin ejecutar; DAST prueba una aplicacion ejecutandose desde fuera.

Que aporta IAST frente a SAST y DAST?

IAST combina ejecucion real con visibilidad interna, por lo que suele dar mas contexto y menos falsos positivos.

Por que es importante combinar herramientas?

Porque cada tecnica ve una parte distinta del problema. SAST encuentra codigo inseguro temprano; DAST encuentra fallos explotables y configuracion real; IAST conecta ejecucion con flujo interno; la monitorizacion observa el sistema en operacion.

Que es un falso positivo?

Una alerta que parece vulnerabilidad pero no lo es en el contexto real.

Que es un falso negativo?

Una vulnerabilidad real que la herramienta no detecta.

Por que DAST puede detectar cosas que SAST no ve?

Porque observa el sistema desplegado: configuracion, cabeceras, TLS, comportamiento de APIs, autenticacion y respuestas reales.

Por que SAST es util en CI/CD?

Porque automatiza revisiones tempranas y evita que patrones inseguros entren en ramas principales o despliegues.

Que problema puede haber si DEV y PROD usan el mismo agente de compilacion?

Un atacante que comprometa una cuenta de desarrollo podria infectar una build DEV y contaminar el agente que despues genera una build de produccion.

Que diferencia hay entre LAS, IPS y GAS?

LAS supervisa comportamiento local de una aplicacion; IPS supervisa interacciones entre aplicaciones dentro de una plataforma; GAS supervisa reglas globales o verticales asociadas a la aplicacion.

10.3 Mapa mental minimo

```
Verificacion y pruebas de seguridad
|
+-- Necesidad
|   +-- amenazas universales
|   +-- cumplimiento legal
|   +-- confianza
|   +-- coste de corregir tarde
|   +-- seguridad por diseno no basta
|
+-- Tipos de pruebas
|   +-- SAST
|   +-- DAST
|   +-- IAST
```

- | +-- Pentesting
- | +-- Fuzzing
- | +-- SCA / dependencias
- | +-- configuracion
- | +-- IAM
- |
- +-- SAST
 - | +-- codigo sin ejecutar
 - | +-- temprano
 - | +-- IDE / CI/CD
 - | +-- falsos positivos
 - |
- +-- DAST
 - | +-- app ejecutandose
 - | +-- vista externa
 - | +-- configuracion y runtime
 - | +-- no localiza tan bien la linea
 - |
- +-- IAST
 - | +-- app ejecutandose + agente
 - | +-- contexto interno
 - | +-- menos ruido
 - |
- +-- Monitorizacion
 - +-- sin supervision
 - +-- autosupervisada
 - +-- instrumentada
 - +-- LAS / IPS / GAS

10.4 Tabla final de ejemplos

Problema	Tecnica que puede detectarlo bien	Correccion tipica
SQL injection por concatenacion	SAST, DAST, IAST	Consultas parametrizadas u ORM
XSS por salida no escapada	SAST, DAST, IAST	Escapado, sanitizacion, CSP
Password hardcodeada	SAST	Variables de entorno o gestor de secretos
debug=True en Flask	SAST	Desactivar debug en produccion

Problema	Tecnica que puede detectarlo bien	Correccion tipica
Deserializacion insegura	SAST, IAST	Evitar datos no confiables o usar formatos seguros
Cabeceras HTTP ausentes	DAST	Hardening y configuracion segura
TLS debil	DAST	Configuracion TLS moderna
Fuerza bruta contra login	DAST, pentesting, monitorizacion	Rate limiting, bloqueo, MFA
Dependencia vulnerable	SCA	Actualizar, sustituir o mitigar
Interaccion inesperada entre apps	Monitorizacion IPS	Reglas de interaccion y reaccion

10.5 Frases clave para recordar

- La seguridad no es una característica añadida al final, es una mentalidad durante todo el ciclo de vida.
- Ningun sistema es demasiado pequeño para necesitar pruebas de seguridad.
- Prevenir vulnerabilidades durante desarrollo es mucho más barato que corregirlas tras el despliegue.
- SAST desplaza la seguridad a la izquierda.
- DAST valida el comportamiento real de la aplicación en ejecución.
- IAST mejora precisión al combinar contexto interno y ejecución real.
- La monitorización permite detectar y reaccionar cuando el sistema ya está funcionando.
- Una estrategia de seguridad robusta combina varias técnicas.